

R - S4 Classes and Methods

Kasper D. Hansen

- [Dependencies](#)
- [Corrections](#)
- [Overview](#)
 - [Important note for programmers](#)
- [Other Resources](#)
- [S3 and S4 classes](#)
- [Constructors and getting help](#)
- [Slots and accessor functions](#)
- [Class inheritance](#)
- [Outdated S4 classes](#)
 - [Notes on class version](#)
- [S4 Methods](#)
- [SessionInfo](#)

Dependencies

This document has the following dependencies:

```
library(ALL)
library(GenomicRanges)
```

Use the following commands to install these packages in R.

```
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("ALL" "GenomicRanges"))
```

Corrections

Improvements and corrections to this document can be submitted on its [GitHub](#) in its [repository](#).

Overview

The S4 system in R is a system for object oriented programming. Confusingly, R has support for at least 3 different systems for object oriented programming: S3, S4 and S5 (also known as reference classes).

The S4 system is heavily used in Bioconductor, whereas it is very lightly used in “traditional” R and in packages from CRAN. As a user it can be useful to recognize S4 objects and to learn some facts about how to explore, manipulate and use the help system when encountering S4 classes and methods.

Important note for programmers

If you have experience with object oriented programming in other languages, for example java, you need to understand that in R, S4 objects and methods are completely separate. You can use S4 classes without every using S4 methods and vice versa.

Other Resources

- The vignette from the [GenomicRanges webpage](#).

S3 and S4 classes

Based on years of experience in Bioconductor, it is fair to say that S4 classes have been very successful in this project. S4 classes has allowed us to construct rich and complicated data representations that nevertheless seems simple to the end user. An example, which we will return to, are the data containers `ExpressionSet` and `SummarizedExperiment`.

Let us look at a S3 object, the output of the linear model function `lm` in base R:

```
df <- data.frame(y = rnorm(10), x = rnorm(10))
lm.object <- lm(y ~ x, data = df)
lm.object
```

```
##
## Call:
## lm(formula = y ~ x, data = df)
##
## Coefficients:
## (Intercept)          x
##      0.3147      -0.7131
```

```
names(lm.object)
```

```
## [1] "coefficients" "residuals" "effects" "rank"  
## [5] "fitted.values" "assign" "qr" "df.residual"  
## [9] "xlevels" "call" "terms" "model"
```



```
class(lm.object)
```

```
## [1] "lm"
```

In standard R, an S3 object is essentially a `list` with a `class` attribute on it. The problem with S3 is that we can assign any class to any list, which is nonsense. Let us try an example

```
xx <- list(a = letters[1:3], b = rnorm(3))  
xx
```

```
## $a  
## [1] "a" "b" "c"  
##  
## $b  
## [1] 0.3380950 0.8861906 -1.1216766
```

```
class(xx) <- "lm"  
xx
```

```
##  
## Call:  
## NULL  
##  
## No coefficients
```

At least we don't get an error when we print it.

S4 classes have a formal definition and formal validity checking. To the end user, this guarantees validity of the object.

Let us load an S4 object:

```
library(ALL)
data(ALL)
ALL
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 12625 features, 128 samples
##   element names: exprs
## protocolData: none
## phenoData
##   sampleNames: 01005 01010 ... LAL4 (128 total)
##   varLabels: cod diagnosis ... date last seen (21 total)
##   varMetadata: labelDescription
## featureData: none
## experimentData: use 'experimentData(object)'
##   pubMedIds: 14684422 16243790
## Annotation: hgu95av2
```

```
class(ALL)
```

```
## [1] "ExpressionSet"
## attr(,"package")
## [1] "Biobase"
```

```
isS4(ALL)
```

```
## [1] TRUE
```

The last function call checks whether the object is S4.

Constructors and getting help

The proper way of finding help on a class is to do one of the following

```
? "ExpressionSet-class"  
class?ExpressionSet
```

Note how you need to put the `ExpressionSet-class` in quotes.

A constructor function is a way to construct objects of the given class. You have already used constructor functions for base R classes, such as

```
xx <- list(a = 1:3)
```

here `list()` is a constructor function. The Bioconductor coding standards suggests that an S4 class should have a name that begins with a capital letter and a constructor function with the same name as the class.

This is true for `ExpressionSet`:

```
ExpressionSet()
```

```
## ExpressionSet (storageMode: lockedEnvironment)  
## assayData: 0 features, 0 samples  
##   element names: exprs  
## protocolData: none  
## phenoData: none  
## featureData: none  
## experimentData: use 'experimentData(object)'  
## Annotation:
```

It is common that the constructor function is documented on the same help page as the class; this is why getting using

```
?ExpressionSet
```

works to give you detail on the class. *This does not always work.*

You can always use the function `new()` to construct an instance of a class. This is now frowned upon in Bioconductor, since it is not a good idea for complicated classes (... years of experience left out here). But in old documents on Bioconductor you can sometimes see calls like

```
new("ExpressionSet")
```

```
## ExpressionSet (storageMode: lockedEnvironment)
## assayData: 0 features, 0 samples
##   element names: exprs
## protocolData: none
## phenoData: none
## featureData: none
## experimentData: use 'experimentData(object)'
## Annotation:
```

An example of a class in Bioconductor that does not have a constructor function is the `BSPARAMS` class from *BSgenome* used for constructing calls to the `bsapply` function (applying functions over whole genomes).

Slots and accessor functions

You can get the class definition as

```
getClass("ExpressionSet")
```

```
## Class "ExpressionSet" [package "Biobase"]
##
## Slots:
##
## Name:      experimentData      assayData      phenoData
## Class:     MIAME               AssayData      AnnotatedDataFrame
##
## Name:      featureData         annotation     protocolData
## Class:     AnnotatedDataFrame   character      AnnotatedDataFrame
##
## Name:      __classVersion__
## Class:     Versions
##
## Extends:
## Class "eSet", directly
## Class "VersionedBiobase", by class "eSet", distance 2
## Class "Versioned", by class "eSet", distance 3
```

In this output you'll see two things

1. A number of `slots` are mentioned together with a name and a `class`.
2. The class “extends” the class `eSet` “directly”.

First, let us discuss (1). Data inside an S4 class are organized into slots. You access slots by using either ‘@’ or the ‘`slots()`’ function, like

```
ALL@annotation
```

```
## [1] "hgu95av2"
```

```
slot(ALL, "annotation")
```

```
## [1] "hgu95av2"
```

However, as a user you **should never have to access slots directly**. This is important to understand. You should get data out of the class using “accessor” functions. Frequently accessor functions are named as the slot or perhaps `get` and the slot name.

```
annotation(ALL)
```

```
## [1] "hgu95av2"
```

(the `get` version of this name is `getAnnotation()` - different package authors use different styles). Not all slots have an accessor function, because slots may contain data which is not useful to the user.

Traditionally, accessor functions are documented on the same help page as the class itself.

Accessor functions does not always precisely refer to a slot. For example, for `ExpressionSet` we use `exprs()` to get the expression matrix, but there is no slot called `exprs` in the class definition. We still refer to `exprs()` as an accessor function.

By only using accessor functions you are protecting yourself (and your code) against future changes in the class definition; accessor functions should always work.

Class inheritance

Class inheritance is used a lot in Bioconductor. Class inheritance is used when you define a new class which “is almost like this other class but with a little twist”. For example `ExpressionSet` inherits from `eSet`, and when you look at the class definition you cannot easily see a difference. The difference is that `ExpressionSet` is meant to contain expression data and has the `exprs()` accessor.

To make the usefulness of this more obvious, let me describe (briefly) the `MethylSet` class from the *minfi* (which I have authored). This class is very similar to `ExpressionSet` except it contains methylation data. Methylation is commonly measured using two channels “methylation” and “unmethylation” as opposed to the single channel exposed by `ExpressionSet`. Both `ExpressionSet` and `MethylSet` inherits from `eSet` (which actually represents most of the code of these classes) but `ExpressionSet` has a single `exprs()` accessor and `MethylSet` has two methylation accessors `getMeth()` and `getUnmeth()`.

This is useful to know because the documentation for a class might often refer to its parent class. For example, in the documentation for `ExpressionSet` you find the phrase “see `eSet`” a lot.

Outdated S4 classes

It occasionally happens that an S4 class definition gets updated. This might affect you in the following scenario

- You do an analysis in a given version of Bioconductor and you save your objects.
- 6 months later your work has to be revised, but Bioconductor has been updated in the meantime.
- When you load the old object, it doesn’t seem to work.

The solution to this problem is the function `updateObject`. When a programmer updates their class definition, they are supposed to provide an `updateObject` function which will update old objects to new objects. Note the “supposed”, this is not guaranteed to happen, but feel free to report this as a bug if you encounter it.

Usage is easy

```
new_object <- updateObject(old_object)
```

In practice, you tend to not want to keep the `old_object` around so you do

```
object <- updateObject(object)
```


As an added hint, you can always run validity checking on an S4 objects if you think something funny is going on. It should return TRUE:

```
validObject(ALL)
```

```
## [1] TRUE
```

Notes on class version

In the early days of Bioconductor, efforts were made to version S4 classes. This was done in anticipation of changes in class definitions. This actually happens. For example, the `ExpressionSet` class has changed definition at least one time, and at the time of writing, the `SummarizedExperiment` class is undergoing changes to its internal structure between Bioconductor 3.1 and 3.2. It was later realized that we do seldom change class definitions, so the versioning was abandoned. You see debris from this in the `.__classversion__` slot of the `ExpressionSet` class.

S4 Methods

You can think of S4 methods as simple functions. A method is a function which can look at its arguments and decide what to do. One way to mimic a method is by a function definition like the following

```
mimicMethod <- function(x) {  
  if (is(x, "matrix"))  
    method1(x)  
  if (is(x, "data.frame"))  
    method2(x)  
  if (is(x, "IRanges"))  
    method3(x)  
}
```

This function examines the `x` argument and runs different sets of code (`method1`, `method2`, `method3`) depending on which class `x` is.

An example of this is `as.data.frame`.

```
as.data.frame
```

```
## standardGeneric for "as.data.frame" defined from package "BiocGenerics"
##
## function (x, row.names = NULL, optional = FALSE, ...)
## standardGeneric("as.data.frame")
## <environment: 0x7ff6d3e84a40>
## Methods may be defined for arguments: x
## Use showMethods("as.data.frame") for currently available ones.
```



In the output you can see that it is so-called “generic” method involved something called `standardGeneric()`. Don’t be distracted by this lingo; this is just like the `mimicMethod` function defined above. To see `method1`, `method2` etc you do

```
showMethods("as.data.frame")
```

```
## Function: as.data.frame (package BiocGenerics)
## x="ANY"
## x="DataFrame"
## x="GappedRanges"
## x="GenomicRanges"
## x="GroupedIRanges"
## x="Hits"
## x="List"
## x="RangedData"
## x="Ranges"
## x="Rle"
## x="Seqinfo"
## x="Vector"
```

The different values of `x` here are called “signatures”.

Actually, this does not show you the actual methods, it just shows you which values of `x` a method has been defined for. To see the code, do

```
getMethod("as.data.frame", "DataFrame")
```

```
## Method Definition:
##
## function (x, row.names = NULL, optional = FALSE, ...)
## {
##     if (length(list(...)))
##         warning("Arguments in '...' ignored")
##     l <- as(x, "list")
##     if (is.null(row.names))
##         row.names <- rownames(x)
##     if (!length(l) && is.null(row.names))
##         row.names <- seq_len(nrow(x))
##     l <- lapply(l, function(y) {
##         if (is(y, "SimpleList") || is(y, "CompressedList"))
##             y <- as.list(y)
##         if (is.list(y))
##             y <- I(y)
##         y
##     })
##     IRanges.data.frame <- injectIntoScope(data.frame, as.data.frame)
##     do.call(IRanges.data.frame, c(l, list(row.names = row.names),
##         check.names = !optional, stringsAsFactors = FALSE))
## }
## <environment: namespace:S4Vectors>
##
## Signatures:
##         x
## target  "DataFrame"
## defined "DataFrame"
```

Lingo - `as.data.frame` is a generic method. It operates on different signatures (values of `x`) and each signature has an associated method. This method is said to “dispatch” on `x`.

Many Bioconductor packages use S4 classes extensively and S4 methods sparingly; I tend to follow this paradigm. S4 methods are particularly useful when

1. there are many different values of the argument which needs to be handled (like `as.data.frame` above).
2. you want to mimic functions from base R.

The second point is the case for, for example, the *IRanges* and *GenomicRanges* packages. The *IRanges* class looks very much like a standard vector and extensive work has gone into making it feel like a standard vector.

For `as.data.frame`, you can see that the value of this function in base R is not a method, by

```
base::as.data.frame
```

```
## function (x, row.names = NULL, optional = FALSE, ...)  
## {  
##   if (is.null(x))  
##     return(as.data.frame(list()))  
##   UseMethod("as.data.frame")  
## }  
## <bytecode: 0x7ff6d20b6a00>  
## <environment: namespace:base>
```

What happens is that the *BiocGenerics* converts the base R function `as.data.frame` into a generic method. This is what you get notified about when the following is printed when you load *BiocGenerics* (typically as by-product of loading another Bioconductor package such as *IRanges*).

The following objects are masked from 'package:base':

```
Filter, Find, Map, Position, Reduce, anyDuplicated, append,  
as.data.frame, as.vector, cbind, colnames, do.call, duplicated,  
eval, evalq, get, intersect, is.unsorted, lapply, mapply, match,  
mget, order, paste, pmax, pmax.int, pmin, pmin.int, rank, rbind,  
rep.int, rownames, sapply, setdiff, sort, table, tapply, union,  
unique, unlist, unsplit
```

There are drawbacks to methods:

1. It is hard (but not impossible) to get the actual code.
2. The help system can be confusing.
3. They are hard to debug for non-package authors.

We have addressed (1) above. The problem with the help system is that each method of `as.data.frame` may have its own help page, sometimes in different packages. Furthermore, each method may have different arguments.

The correct way to look up a help page for a method is

```
method?as.data.frame,DataFrame  
?"as.data.frame-method,DataFrame"
```

which is quite a mouthful. This becomes worse when there is dispatching on multiple arguments; a great example is

```
showMethods("findOverlaps")
```

```
## Function: findOverlaps (package IRanges)
## query="GenomicRanges", subject="GenomicRanges"
## query="GenomicRanges", subject="GIntervalTree"
## query="GenomicRanges", subject="GNCList"
## query="GenomicRanges", subject="GRangesList"
## query="GenomicRanges", subject="RangedData"
## query="GenomicRanges", subject="RangesList"
## query="GNCList", subject="GenomicRanges"
## query="GRangesList", subject="GenomicRanges"
## query="GRangesList", subject="GRangesList"
## query="GRangesList", subject="RangedData"
## query="GRangesList", subject="RangesList"
## query="integer", subject="Ranges"
## query="NCList", subject="Ranges"
## query="RangedData", subject="GenomicRanges"
## query="RangedData", subject="GRangesList"
## query="RangedData", subject="RangedData"
## query="RangedData", subject="RangesList"
## query="Ranges", subject="IntervalTree"
## query="Ranges", subject="NCList"
## query="Ranges", subject="Ranges"
## query="RangesList", subject="GenomicRanges"
## query="RangesList", subject="GRangesList"
## query="RangesList", subject="IntervalForest"
## query="RangesList", subject="RangedData"
## query="RangesList", subject="RangesList"
## query="SummarizedExperiment", subject="SummarizedExperiment"
## query="SummarizedExperiment", subject="Vector"
## query="Vector", subject="missing"
## query="Vector", subject="SummarizedExperiment"
## query="Vector", subject="Views"
## query="Vector", subject="ViewsList"
## query="Views", subject="Vector"
## query="Views", subject="Views"
## query="ViewsList", subject="Vector"
## query="ViewsList", subject="ViewsList"
```

Finding the right help page for a method is (in my opinion) currently much harder than it ought to be; console yourself that many people struggle with this.

findoverlaps is also an example where two different methods of the generic have different arguments, although it becomes extremely confusing to illustrate how findoverlaps only accepts ignore.strand when the argument is a GRanges and not an IRanges. You cannot see it in the method arguments; you need to read the code itself (or the help page):

```
getMethod("findOverlaps", signature(query = "Ranges", subject = "Ranges")
```

```
## Method Definition:
##
## function (query, subject, maxgap = 0L, minoverlap = 1L, type = c("any",
##   "start", "end", "within", "equal"), select = c("all", "first",
##   "last", "arbitrary"), algorithm = c("nclist", "intervaltree"),
##   ...)
## {
##   .local <- function (query, subject, maxgap = 0L, minoverlap = 1L,
##     type = c("any", "start", "end", "within", "equal"), select =
##       "first", "last", "arbitrary"), algorithm = c("nclist",
##         "intervaltree"))
##   {
##     type <- match.arg(type)
##     select <- match.arg(select)
##     algorithm <- match.arg(algorithm)
##     if (algorithm == "intervaltree") {
##       subject <- IntervalTree(subject)
##       findOverlaps(query, subject, maxgap = maxgap, minoverlap
##         type = type, select = select)
##     }
##     else {
##       findOverlaps_Ranges(query, subject, maxgap = maxgap,
##         minoverlap = minoverlap, type = type, select = select)
##     }
##   }
##   .local(query, subject, maxgap, minoverlap, type, select,
##     algorithm, ...)
## }
## <environment: namespace:IRanges>
##
## Signatures:
##       query      subject
## target  "Ranges" "Ranges"
## defined "Ranges" "Ranges"
```

```
getMethod("findOverlaps", signature(query = "GenomicRanges", subject = "
```

```
## Method Definition:
##
## function (query, subject, maxgap = 0L, minoverlap = 1L, type = c("any",
##   "start", "end", "within", "equal"), select = c("all", "first",
##   "last", "arbitrary"), algorithm = c("nclist", "intervaltree"),
##   ...)
## {
##   .local <- function (query, subject, maxgap = 0L, minoverlap = 1L,
##     type = c("any", "start", "end", "within", "equal"), select =
##     "first", "last", "arbitrary"), algorithm = c("nclist",
##     "intervaltree"), ignore.strand = FALSE)
##   {
##     type <- match.arg(type)
##     select <- match.arg(select)
##     algorithm <- match.arg(algorithm)
##     if (algorithm == "nclist")
##       FUN <- findOverlaps_GenomicRanges
##     else FUN <- findOverlaps_GenomicRanges_old
##     FUN(query, subject, maxgap = maxgap, minoverlap = minoverlap,
##       type = type, select = select, ignore.strand = ignore.strand)
##   }
##   .local(query, subject, maxgap, minoverlap, type, select,
##     algorithm, ...)
## }
## <environment: namespace:GenomicRanges>
##
## Signatures:
##       query      subject
## target  "GenomicRanges" "GenomicRanges"
## defined "GenomicRanges" "GenomicRanges"
```

This is (in some ways) a great illustration of how confusing methods can be! The good thing is that they tend to “just work”.

SessionInfo

```
## R version 3.2.1 (2015-06-18)
## Platform: x86_64-apple-darwin13.4.0 (64-bit)
## Running under: OS X 10.10.5 (Yosemite)
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## attached base packages:
## [1] stats4      parallel  methods    stats      graphics  grDevices  utils
## [8] datasets   base
##
## other attached packages:
## [1] GenomicRanges_1.20.5 GenomeInfoDb_1.4.2  IRanges_2.2.7
## [4] S4Vectors_0.6.3      ALL_1.10.0          Biobase_2.28.0
## [7] BiocGenerics_0.14.0  BiocStyle_1.6.0     rmarkdown_0.7
##
## loaded via a namespace (and not attached):
## [1] digest_0.6.8    formatR_1.2      magrittr_1.5     evaluate_0.7.2
## [5] stringi_0.5-5   XVector_0.8.0    tools_3.2.1      stringr_1.0.0
## [9] yaml_2.1.13     htmltools_0.2.6  knitr_1.11
```